

TensorFlow Implementation

Inference in Code

Let's dive in. One of the remarkable things about neural networks is the same algorithm can be applied to so many different applications.

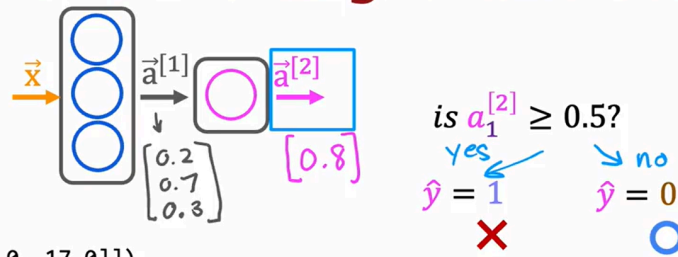
For this video and in some of the labs for you to see what the neural network is doing, I'm going to use another example to illustrate inference. Sometimes I do like to roast coffee beans myself at home. My favorite is actually Colombian coffee beans.

Can the learning algorithm help optimize the quality of the beans you get from a roasting process like this? When you're roasting coffee, two parameters you get to control are the temperature at which you're heating up the raw coffee beans to turn them into nicely roasted coffee beans, as well as the duration or how long are you going to roast the beans. In this slightly simplified example, we've created the datasets of different temperatures and different durations, as well as labels showing whether the coffee you roasted is good-tasting coffee. Where cross here, the positive cross $y = 1$ corresponds to good coffee, and all the negative cross corresponds to bad coffee.

It looks like a reasonable way to think of this dataset is if you cook it at too lower temperature, it doesn't get roasted and it ends up undercooked. If you cook it, not for long enough, the duration is too short, it's also not a nicely roasted set of beans. Finally, if you were to cook it either for too long or for too higher temperature, then you end up with overcooked beans. They're a little bit burnt beans. There's not good coffee either.

It's only points within this little triangle here that corresponds to good coffee. This example is simplified a bit from actual coffee roasting. Even though this example is a simplified one for the purpose of illustration, there have actually been serious projects using machine learning to optimize coffee roasting as well.

Build the model using TensorFlow



```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
```

```
layer_2 = Dense(units=1, activation='sigmoid')
a2 = layer_2(a1)
```

```
if a2 >= 0.5:
    yhat = 1
else:
    yhat = 0
```

The task is given a feature $\vec{x}$ with both temperature and duration, say 200 degrees Celsius for 17 minutes, how can we do inference in a neural network to get it to tell us whether or not this temperature and duration setting will result in good coffee or not?

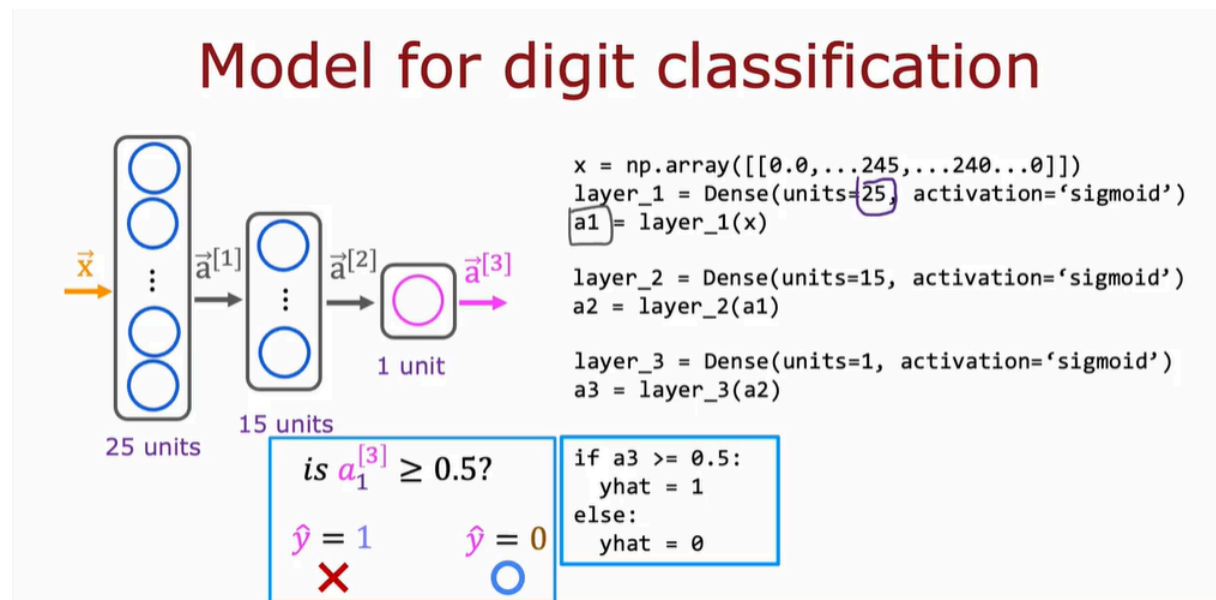
It looks like this. We're going to set x to be an array of two numbers. The input features 200 degrees celsius and 17 minutes. Then you create Layer 1 as this first hidden layer, the neural network, as dense open parenthesis units 3, that means three units or three hidden units in this layer using as the activation function, the sigmoid function.

Dense is another name for the layers of a neural network that we've learned about so far. As you learn more about neural networks, you learn about other types of layers as well. But for now, we'll just use the dense layer, which is the layer type you've learned about in the last few videos for all of our examples.

Next, you compute $\vec{a}^{[1]}$ by taking Layer 1, which is actually a function, and applying this function Layer 1 to the values of x . That's how you get $a1$, which is going to be a list of three numbers because Layer 1 had three units. So $\vec{a}^{[1]}$ here may, just for the sake of illustration, be 0.2, 0.7, 0.3. Next, for the second hidden layer, Layer 2, would be dense. Now this time it has one unit and again to sigmoid activation function, and you can then compute $\vec{a}^{[2]}$ by applying this Layer 2 function to the activation values from Layer 1 to $\vec{a}^{[1]}$. That will give you the value of $\vec{a}^{[2]}$, which for the sake of illustration is maybe 0.8.

Finally, if you wish to threshold it at 0.5, then you can just test if $\vec{a}^{[2]} \geq 0.5$ and set $\hat{y}$ equals to one or zero positive or negative cross accordingly. That's how you do inference in the neural network using TensorFlow. There are some

additional details that I didn't go over here, such as how to load the TensorFlow library and how to also load the parameters w and b of the neural network. But we'll go over that in the lab. Please be sure to take a look at the lab. But these are the key steps for forward propagation in how you compute $\vec{a}^{[1]}$ and $\vec{a}^{[2]}$ and optionally threshold $\vec{a}^{[2]}$.



Dense is like a layer

Data in TensorFlow

Feature vectors

temperature (Celsius)	duration (minutes)	Good coffee? (1/0)
200.0	17.0	1
425.0	18.5	0
...	...	...

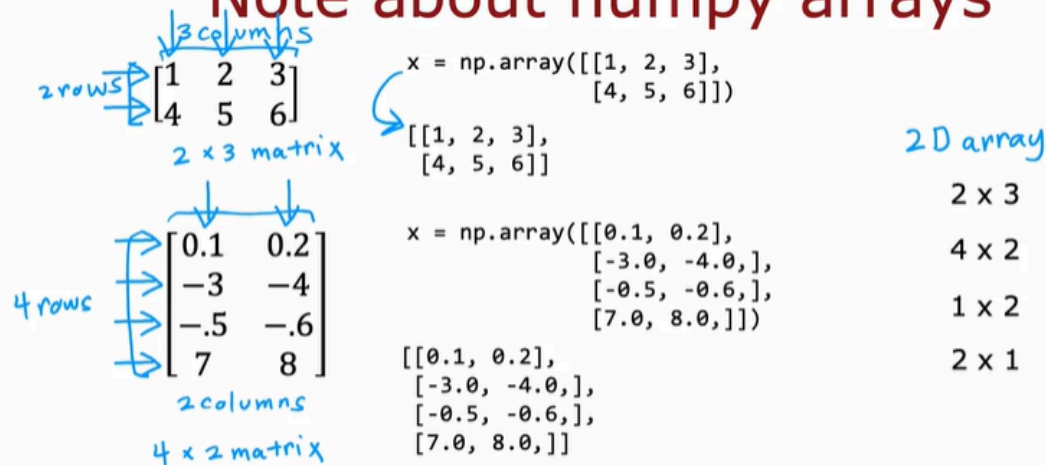
$x = \text{np.array}([[200.0, 17.0]])$ ←

$[[200.0, 17.0]]$

why?

Matrices can have different dimensions.

Note about numpy arrays



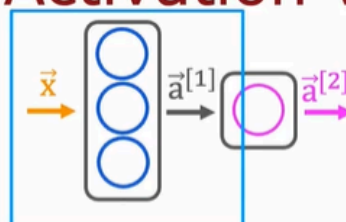
if we use:

```
x = np.array([200, 12]) # this is a vector → a linear array, no rows, no columns
##
x = np.array([[200, 17]]) # this is a matrix
```

With TensorFlow the convention is to use matrices to represent the data

- TensorFlow was designed to handle very large datasets and by representing the data in matrices instead of 1D arrays, it lets TensorFlow be a bit more computationally efficient internally.

Activation vector



```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
→ [[0.2, 0.7, 0.3]] 1 x 3 matrix
→ tf.Tensor([[0.2 0.7 0.3]], shape=(1, 3), dtype=float32)
→ a1.numpy()
array([[0.2, 0.7, 0.3]], dtype=float32)
```

Once again you can convert from a tensorflow tensor to a NumPy matrix using `a2.numpy` and that will turn this back into a NumPy array that looks like this. So that hopefully gives you a sense of how data is represented in TensorFlow and in NumPy.

Building a neural network

Building a neural network architecture

```

→ model = Sequential([
→   Dense(units=3, activation="sigmoid"),
→   Dense(units=1, activation="sigmoid")])

x = np.array([[200.0, 17.0],
              [120.0, 5.0],
              [425.0, 20.0],
              [212.0, 18.0]])
# 4 x 2

y = np.array([1,0,0,1])
# targets

model.compile(...) ← more about this next week!
model.fit(x,y)
→ model.predict(x_new) ←
  
```

		y
200	17	1
120	5	0
425	20	0
212	18	1

We can instead tell tensorflow that we would like it to take layer one and layer two and string them together to form a neural network. That's what the `sequential` function in TensorFlow does which is it says, Dear TensorFlow, please create a neural network for me by sequentially string together these two layers that I just created. It turns out that with the sequential framework tensorflow can do a lot of work for you.

For efficiency, the training data set is broken into 'batches'. The default size of a batch in Tensorflow is 32. There are 200000 examples in our expanded data set or 6250 batches (200000 / 32).

Code in labs!!!!